

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

LayerIt!: Towards a Time-Aligned Framework for Composable Music Visualisation Layers

<Fernando Azeredo>

WORKING VERSION



Mestrado em Engenharia Informática e Computação

Supervisor: Prof. <António Sá Pinto>

July 22, 2026

Abstract

Music Information Retrieval (**MIR**) visualization tools play a crucial role in music analysis, annotation, and evaluation, but existing solutions often make it difficult to combine audio, symbolic scores, and algorithmic outputs in a single, time-aligned view. This thesis presents LayerIt, a Python-based object-oriented (Object-Oriented (**OO**)) framework for generating custom score-aligned **MIR** visualizations by combining established libraries and tools, including librosa, matplotlib, Modusa, ScoreWarp, and BeatThis. LayerIt is built around a modular layer-composition model in which data sources, audio plots, score renderings, and algorithmic outputs are treated as independent components that can be assembled into tailored visualizations. The resulting figures are exported as Scalable Vector Graphics (**SVG**), enabling resolution-independent output, post-processing, and future integration with web-based or interactive applications. As a validation case, LayerIt is used to visualize beat tracking results alongside aligned score and audio representations, showing how the framework supports both beat annotation and beat tracking evaluation through discrete beat estimates and continuous confidence curves. More broadly, the project demonstrates a reusable foundation for future specialized **MIR** visualization workflows.

Keywords: Music Information Retrieval • Visualization • Audio-to-Score Alignment • Beat Tracking • SVG • Object-Oriented • Python

Acknowledgements

<Fernando Azeredo>

Contents

1	Introduction	1
1.1	Contextualization	1
1.2	Motivation	1
1.3	Objectives, Research Questions and Contributions	2
1.4	Developed Project	3
1.5	Thesis Structure	3
2	Literature Review	5
2.1	Visualisation Types	5
2.1.1	Sheet Music and Score Images	5
2.1.2	Symbolic Representations	6
2.1.3	Audio Representations	8
2.2	MIR Visualisation Tools	13
2.3	Audio-to-Score Alignment	13
2.4	Summary	15
3	Methodology	16
3.1	Research Approach	16
3.2	Iterative Development Process	17
3.3	Scope Decision	19
3.4	Evaluation Criteria	19
4	LayerIt!	
	Design and Implementation	21
4.1	The Problem and our Solution	21
4.2	LayerIt Overview	21
4.3	Design Choices and Rationale	22
4.4	Generating Visuals	23
4.4.1	Getting the Warped Score	23
4.4.2	Getting the MAPS file	23
4.4.3	Providing BT data	24
4.5	Layer Alignment Method	25
4.6	Layer Alignment Method	25
4.7	Layers on Layers, within Layers	26
4.7.1	Visual Level	27
4.7.2	SVG Level	27
4.7.3	Code Level	27
4.8	Summary	30

5	Validation	31
5.1	Validation Approach	31
5.2	Reproducing Composite Figure Types (RQ1)	31
5.3	Workflow Comparison: Annotation and Evaluation (RQ2, RQ3)	31
5.4	Extensibility Probe (RQ2)	32
5.5	Limitations and Difficult Cases (RQ2, RQ3)	32
5.6	Summary	32
6	Conclusions and Future Work	33
6.1	Achievements	33
6.2	Future Work	33
6.3	Final Remarks	33
	References	34

List of Figures

2.1	Example of an audio waveform.	8
2.2	Example of audio Spectrum.	9
2.3	Example of Linear spectrogram.	10
2.4	Example of Logarithmic spectrogram.	10
2.5	Example of Mel spectrogram.	11
2.6	Example of a CQT.	12
2.7	Example a Chromagram.	12
4.1	Composition of LayerIt visualization system. Each colored box represents a distinct visual element that compose higher-level visualizations.	26
4.2	Complete LayerIt! workflow: from Visualizer() composition to final stacked SVG visualization. The diagram shows how multiple visualization layers are created independently, exported to SVG format, combined with the warped score, and finally stacked vertically with defined y-offsets to produce the final custom composite visualization.	29
5.1	LayerIt generated visualization example for comparing the end output of BeatThis (on top) and the probability layer (at the bottom). In blue downbeats, in red beats and the dotted white lines show onsets.	31

List of Tables

3.1	Summary of development iterations	18
4.1	Required files for LayerIt	23

List of Acronyms

GUI	Graphical User Interface
MEI	Music Encoding Initiative
MIDI	Musical Instrument Digital Interface
MIR	Music Information Retrieval
OO	Object-Oriented
SVG	Scalable Vector Graphics

Chapter 1

Introduction

1.1 Contextualization

MIR is a research field that develops computational methods to analyze, interpret, and retrieve musical information from performance recordings and symbolic representations. MIR bridges a wide range of disciplines, including musicology, signal processing, machine learning, and human-computer interaction. As a data-intensive field, visualization plays a crucial role in MIR research, the how and what is visualized, shapes the conclusions we draw and guides further progress.

The challenge of visualization in MIR is particularly significant because music is an inherently abstract and subjective medium. Translating music into the visual domain is a problem that spans millennia; throughout history, multiple systems of musical notation have been developed in attempts to capture sound in written form, yet this remains an evolving challenge. For MIR purposes, this means that musical data comes in a variety of forms: from audio recordings, to symbolic scores, to annotations of expressive qualities. Each of these require different visual approaches. An audio visualization such as a waveform or spectrogram reveals how a performance sounds, while a musical score conveys how a piece should be performed; both serve distinct and important purposes for different research tasks.

1.2 Motivation

MIR research increasingly demands visualizations that integrate multiple modalities simultaneously. MIR researchers often need to inspect algorithmic outputs alongside audio representations and symbolic scores in a time-coherent manner. This kind of multi-modal, time-aligned visualization is central to both the evaluation of MIR algorithms and the development of annotation workflows. Audio-to-Score Alignment (i.e., the task of synchronizing a performance recording with its corresponding symbolic score) is a key enabler of this type of visualization, as it provides the temporal anchor needed to align heterogeneous data sources into a unified view.

This thesis was initiated by two closely related challenges within Beat Tracking (BT) and Beat Annotation (BA) research. Contemporary BT systems are predominantly deep learning-based

thus, the field is deeply reliant on tools to enable better execution of annotation and evaluation tasks, these are essential for advancing BT research.

Annotation tasks, and more concretely BA itself, remain a laborious and time-consuming process. Existing Graphical User Interface (GUI) tools such as Sonic Visualizer present a clear gap in this area: integrating newer state-of-the-art BT algorithms is difficult, and BA specific features have seen little meaningful innovation. This disconnect between annotation workflows and the algorithms they are meant to support is a concrete obstacle to progress in both domains.

For evaluation tasks, GUI tools suffer from a similar problem, where methods to visualize how developed algorithms perform on a deeper level are lacking. Providing better tools to evaluate and compare models between each other is also key to improving them.

Although solutions like Sonic Visualizer are powerful and widely used, their development cannot keep pace with the modern MIR research needs. Integrating new algorithms or producing specialized visualizations is often impractical within them. A particularly under served case is the visualization of score-aligned data: combining audio and data representations with a musical score typically involves manual image manipulation, with no unified methodology in the community.

Because of these reasons, researchers frequently resort to *ad hoc* solutions that lack reproducibility and are difficult to share or extend, a fragmentation that represents a real obstacle to the collaboration which MIR research depends on. By contributing a flexible, extensible, and community-oriented visualization tool built on established libraries, this thesis aims to not only address an immediate practical gap, but also to establish a foundation for a framework that can support the growing complexity and interdisciplinarity of MIR research.

All of the concerns described in this section serve as a basis for the objectives and research questions we explicitly present in the following section.

1.3 Objectives, Research Questions and Contributions

This thesis is guided by three objectives, which in turn lead to three research questions and a set of concrete contributions.

- **Objective 1:** Develop a reusable Python framework for composing score-aligned MIR visualizations based on an object-oriented layer-composition model.
- **Objective 2:** Demonstrate that the framework supports visualization workflows that are difficult or impractical with current MIR tools.
- **Objective 3:** Assess the usefulness of the framework for beat annotation and beat-tracking evaluation workflows.
- **RQ1:** How can heterogeneous MIR visualizations be combined into a common score-aligned representation?

- **RQ2:** To what extent does the proposed framework simplify the creation, modification, and extension of score-aligned visualizations compared with manual SVG editing, Sonic Visualizer, and Piano Precision?
- **RQ3:** How can score-aligned visualizations support beat annotation and beat-tracking evaluation workflows, and under what conditions does that support break down?

The main contributions of this thesis are the following:

- The LayerIt framework for composing score-aligned MIR visualizations through an object-oriented layer-composition model.
- The release of LayerIt as an open-source, pip-installable Python library.
- The open-source repository containing the five prototype visualizations developed as part of this work.
- A Late-Breaking Demo paper submitted to ISMIR 2026.

The contribution is therefore the layer-composition framework and the figures it enables, rather than the alignment method itself, which is provided by ScoreWarp.

1.4 Developed Project

To address the previously mentioned objectives and RQ, we developed LayerIt!, a Python-based object-oriented framework for composing score-aligned MIR visualizations. LayerIt integrates established MIR libraries and tools into an SVG-based workflow in which visual elements are treated as independent layers that can be composed, reordered, and combined into custom figures.

At its core, LayerIt supports the integration of symbolic scores with audio and algorithmic data in a single view. By relying on ScoreWarp for temporal alignment, it makes it possible to overlay spectrograms, onset information, and beat-tracking outputs on top of score representations within the same visualization. The result is a flexible layer-composition model that supports the prototypes developed in this thesis and provides a foundation for future MIR visualization workflows.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a literature review covering the main audio visualization types, along with existing Graphical User Interface (GUI) and code-based visualization tools, and the task of Audio-to-Score Alignment.
- **Chapter 3** presents the adopted methodology in detail. Followed by a description of the five iterative phases the project took before culminating in the development of LayerIt!.

- **Chapter 4** describes the design and implementation of LayerIt, everything from a detailed description of the problem and solution we developed along side with some broad technical descriptions and the crucial “layers within layers” philosophy that underlines LayerIt!’s design and usage.
- **Chapter 5** is dedicated to the describing the validation of this thesis and project. This chapter should give some closure to the RQ that are outlined in this introduction chapter.
- **Chapter 6** reflects on the achievements of the thesis and outlines directions for future work.

Chapter 2

Literature Review

In this chapter, we will present the required background contextualization for this thesis. We begin by exploring the landscape of music visualisation methods, categorizing them into three main types: sheet music representations, symbolic machine-readable formats, and audio-based visualisations. This foundation is essential for understanding how different types of musical information can be represented visually and interacted with computationally. We then examine contemporary MIR visualisation tools, distinguishing between GUI-based solutions and code-based workflows to highlight the gap that motivates our work. Finally, we introduce Audio-to-Score Alignment, and explore relevant tools and approaches that have informed our solution.

2.1 Visualisation Types

We can sum up all music related visualisations into three categories:

1. Sheet Music and Score Images
2. Symbolic Representations
3. Audio Representations

2.1.1 Sheet Music and Score Images

Sheet music describes musical work using a formal language based on musical symbols and letters, thus, this kind of music representations always require a level of music literacy from the performing musician. Furthermore, sheet music usually does not induce explicit information on how a given piece should be performed, it is intended for the musician to have a certain level of interpretation liberty. This can include the variance of tempo, articulation and dynamics among others.

Sheet music also comes in a variety of forms that vary throughout History and cultures. For our purposes we are going to focus on the western standard of music notation. Still, within western music, there can be vastly different forms of notation, however, most are based on a staff which

sets five horizontal lines and four spaces, each representing a different musical pitch. Musical relevant symbols are put inside the staff to denote pitch, temporal and expressive indications. [10]

2.1.2 Symbolic Representations

Symbolic Representations are machine-readable formats where musical events and expressions are explicitly encoded. These come in various formats, such as Musical Instrument Digital Interface (MIDI) and MusicXML.

Visually we can identify two main applications for this category, these include: piano-roll representations and digital score representations.

Historically piano-roll representations come from so-called “player pianos”, these were self playing pianos that became quite popular in the late 19th to early 20th century. This was achieved by a mechanism that would hit keys of the piano through a continuous roll of paper with perforations. The roll would move over a reading system (known as a tracker bar) that would trigger the musical actions accordingly.

This concept of a continuous and explicit representation of music performance would later prove useful for digital applications. Such is the case for the Piano-Roll.

Today the **Piano-Roll** is a staple feature on any Digital Audio Workstation (DAW) as a visual and interactive tool to write and edit musical events in the form of Music Information Digital Interface (MIDI) notes and attributes (such as velocity and other expressive controls).

MusicXML refers to the XML based language to transcribe score music into the digital domain. There are multiple sub-variants of the MusicXML format for specific applications and there are other formats for digital scores, but MusicXML can be considered as a "universal" format accepted in most digital score editors.

MusicXML was developed to serve as a universal format for storing and sharing music scores across different music notation applications. Music notation digital editors then employ proprietary variants of this format for addressing the program’s specific characteristics. In some cases these may not even be xml based, but MusicXML format can be considered the universal format, generally accepted and exportable in everyone. These include, among others:

- **.mscz and .mscx**: These are MuseScore specific formats named for "MuseScore Format" and "Uncompressed MuseScore" formats respectively. Both xml based.
- **.dora** - "Dorico" specific format (non xml based)
- **.sib** - "Sibelius" proprietary format of company Avid (also non xml based)
- **.mei** - The MEI format, XML based.

The .mei format is the one that is essential for this thesis and the most useful for MIR purposes. MEI stands for both the XML-based score format in itself (.mei), and the community that develops it (Music Encoding Initiative). Citing from the MEI website’s introduction:

“The Music Encoding Initiative (MEI) is a community-driven, open-source effort to define a system for encoding musical documents in a machine-readable structure. MEI brings together specialists from various music research communities, (...) to define best practices for representing a broad range of musical documents and structures.”[1]

Important to note that MEI focuses on a "machine-readable structure". This structure relies on unique identifiers (@xml:id) that make every element in the score individually addressable, from individual noteheads to complex spanning elements like slurs and ties. However, more than a music score format, MEI also serves to keep various notation, performance and interpretation data not only stored but also linked to individual score elements. This is why the MEI format and its community are so essential to MIR as a whole.

That being said, MEI is a purely data driven project, and thus in order for it to be visualized or interacted with it needs to be decoded into a "human-readable format". That's where Verovio comes in. The MEI and Verovio projects are closely inter-linked for that reason. While MEI defines how musical information is stored, Verovio is the primary tool used to transform that data into interactive visualisations and performance-ready outputs.

Verovio is a comprehensive open-source library designed to serve as a native typesetting engine for the MEI format. Verovio's primary role is to engrave MEI data directly into SVG without losing rich metadata through intermediate format conversions. The usage of the SVG format is an essential one for multiple reasons, quoting from Verovio's reference book:

“In a web environment, the addressability can be used for highlighting graphical elements such as notes or any other music symbols. One additional characteristic of SVG is that its XML tree can be structured as desired, and an innovative design feature of Verovio was to go further in the structuring of the output by leveraging this characteristic. Since Verovio implements the MEI structure internally, this key feature of SVG made it possible to preserve the MEI structure in the design of the SVG output in Verovio. Preserving the MEI structure in the SVG output is a considerable overhead in the rendering process but makes it a unique feature of Verovio.

As a result, Verovio output in SVG is not the end of a unidirectional rendering process. Quite on the contrary, it should instead be seen as an intermediate layer standing between the MEI encoding and its rendering that can act as the cornerstone for a bi-directional interaction: from the encoding to the notation, but also from the notation to the encoding through the user interface.” [13]

To further demonstrate the usefulness of MEI and Verovio, we'll present next a quote from the article relative to Piano Precision: a GUI software for audio-to-score visualisation. We will later explore this tool further, for now the importance of this quote comes from demonstrating a practical application of MEI and Verovio.

“Technically, what makes MEI especially useful is that it can be rendered to graphical SVG output while retaining its semantically meaningful XML hierarchy

and unique identifiers for every score element (e.g., ties, notes). Piano Precision renders the scores into SVG output using Verovio, an open-source toolkit for visualizing MEI data. This makes it easy to locate interesting objects in a score and, e.g., highlight them in the rendering. The result is interactive images in which users can flip pages, zoom in and out of a score, and click on elements in a score to jump to their playback positions. ” [5]

2.1.3 Audio Representations

Audio representations are graphs and/or plots that intend to represent sound. These may not be for strictly representing music, the intent is to translate sound from the audible to the visual domain. Audio is a term that refers to an acoustic sound that is turned into an electrical signal. Most audio representations, are thus digitally generated through applying mathematical equations to the audio signal, such as the Fourier Transform.

We can infer at least 3 components that define a given audio. These are, time, frequency and amplitude/energy. An audio visualisation will present explicit information of at least 2 of these.

Waveforms are the most common and one of the simplest ways to represent audio. It consists of a single line on a graph in a 2D plot where amplitude is the y-axis and time is in the x-axis. This representation does not provide frequency information, it only shows how amplitude of the audio changes through time.

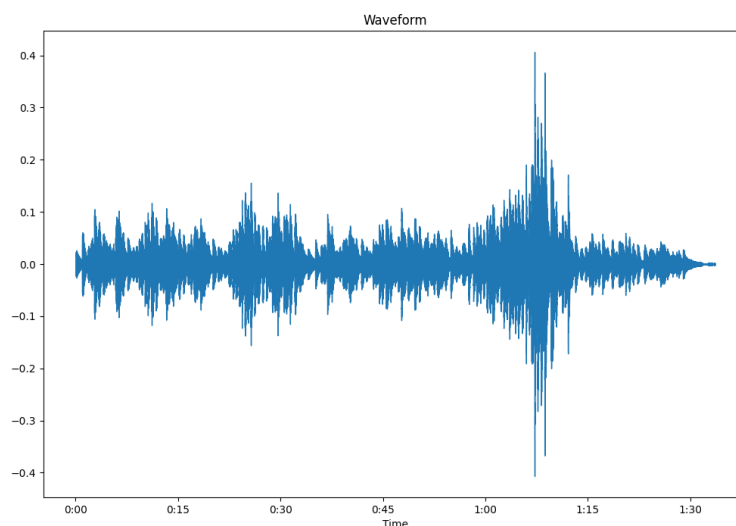


Figure 2.1: Example of an audio waveform.

A spectral visualisation, or a spectrum, refers to a graph with only frequency and amplitude/energy representation in a given time window.

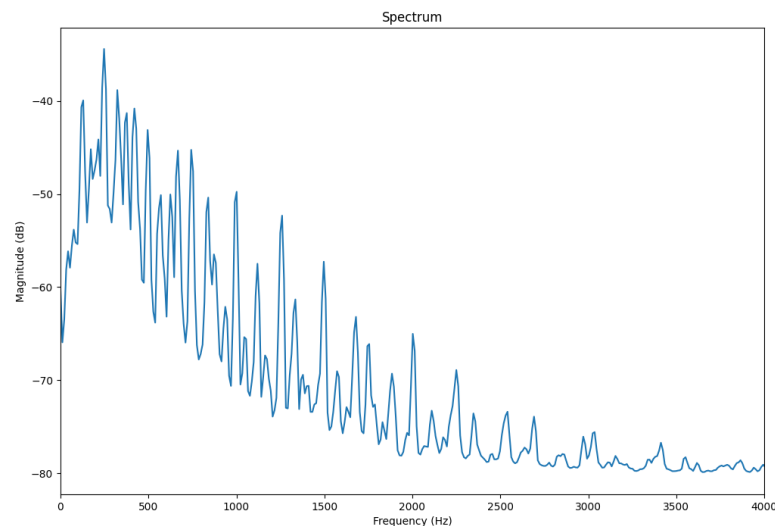


Figure 2.2: Example of audio Spectrum.

For displaying the three sound components the plot needs to represent three distinct dimensions. The spectrogram achieves this by having on the y axis the frequency, time on the x-axis and color as substitute for the third axis (amplitude), and in this way displaying all three parameters in a single 2D visual plot. There are multiple weights and variances of the spectrogram that have different applications.

The way frequencies are displayed in a given spectrogram can help understanding audio better for certain situations. There can be a wide variety of spectrogram representations, that can vary in how it's contents are displayed.

The linear spectrogram is the most basic form of spectrogram, where frequencies are displayed in a linear scale. Meaning, frequencies are equally spaced on the y-axis. For music applications, this kind of representation tends to be less useful, as it does not reflect the way humans perceive sound.

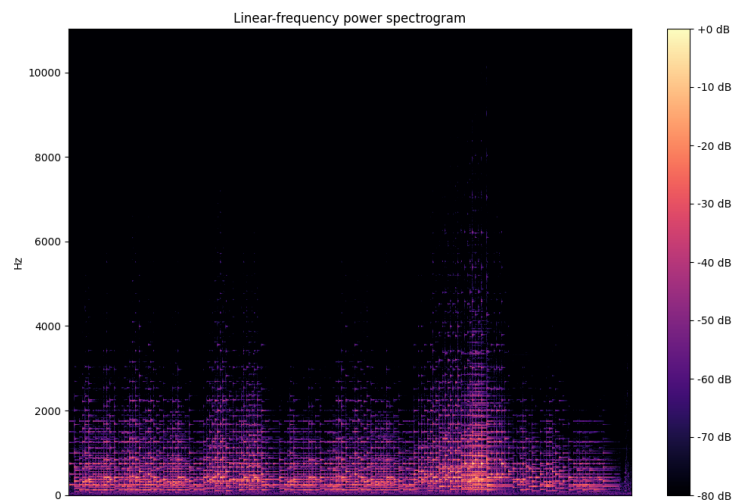


Figure 2.3: Example of Linear spectrogram.

The logarithmic spectrogram displays frequencies in a logarithmic scale. This mimics better the way humans perceive sound.

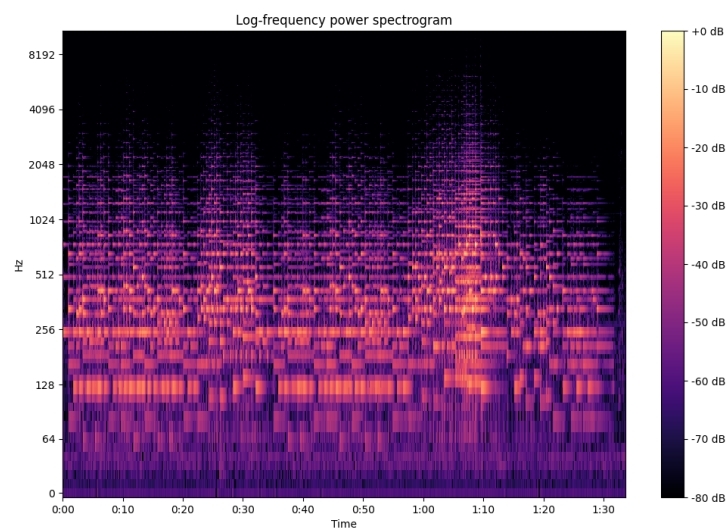


Figure 2.4: Example of Logarithmic spectrogram.

The Mel spectrogram also displays frequencies taking into account the human hearing curve. The difference being, that the logarithmic spectrogram simply organizes frequencies logarithmically. Whilst the Mel spectrogram organizes frequencies according to the Mel scale, which is a perceptual scale of pitches judged by listeners to be equal in distance from one another. For music related applications this type of spectrogram is much more useful, since we can more clearly distinguish in between musical pitches.

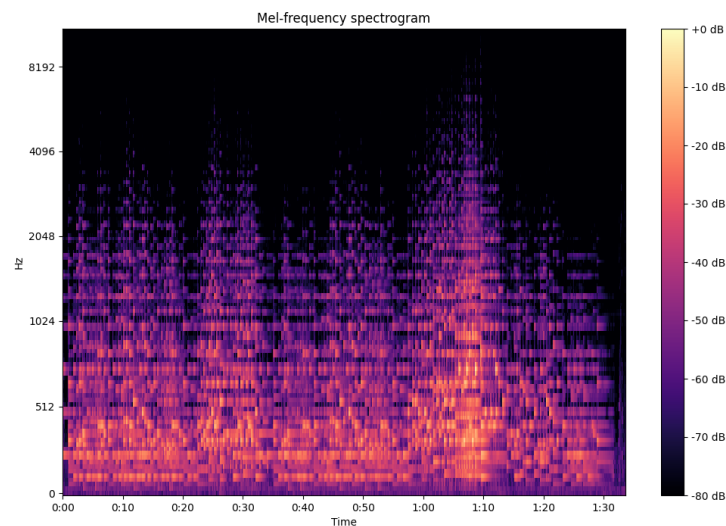


Figure 2.5: Example of Mel spectrogram.

Next I'll introduce the Constant-Q Transform (CQT) and the Chromagram. These are, to some extent, visual representations that derive from the spectrogram, the difference being that whilst the later shows frequency data, the CQT and chromagram can be used to visualize note data. To be exact, tone height in the case of the CQT and Chroma for the Chromagram. The following quote explains these terms accurately:

“(...) a pitch can be separated into two components, which are referred to as tone height and chroma. The tone height refers to the octave number and the chroma to the respective pitch spelling attribute contained in the set {C, C $\sharp$, D,(...), B}.” [10]

The CQT organizes its frequency content using geometrically spaced frequency bins. This is made in order to resemble the equal-tempered system. More commonly, the CQT can be interpreted as a Piano-Roll like representation.[15, p. 3]

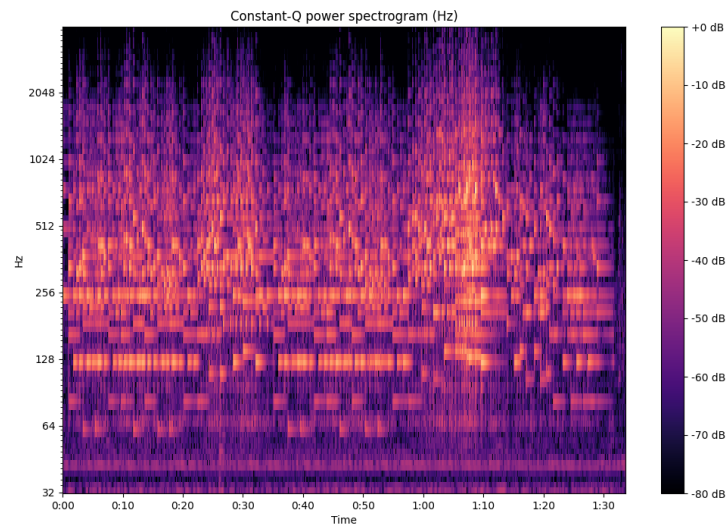


Figure 2.6: Example of a CQT.

While the CQT can distill the frequency spectrum into bins that correlate to individual notes, the chromagram takes this a step further by visually summing the energy of each note across all octaves. Thus in the end we get a plot divided into 12 lines, each representing a give note. In it's essence this type of visualisation serves to see the usage of the same notes across the recording. A chromagram can be useful for determining harmonic and scale related data.

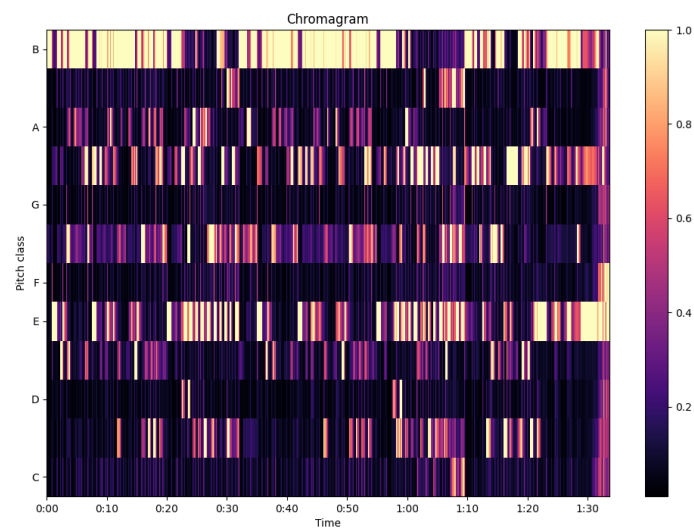


Figure 2.7: Example a Chromagram.

[6, 7, 10, 15]

2.2 MIR Visualisation Tools

Within MIR, visualisation workflows typically fall into two broad categories: GUI-based tools and programming-based workflows. For this thesis, the most relevant comparison is between Sonic Visualizer as a representative GUI tool and code-based workflows used for custom MIR analysis.

Sonic Visualizer is a dedicated program for audio visualisation and analysis, and it is one of the most common tools in MIR practice. It is well suited to inspection, annotation, and exploratory analysis, making it a strong baseline for research-oriented visual work. A particularly relevant example is Piano Precision, a Sonic Visualizer based GUI tool for visualizing score-aligned spectrograms.

“The Piano Precision UI is adapted from Sonic Visualizer, with an additional area displaying a score (...) The interface is designed to help streamline the processes of loading a digital score and a performance recording, editing and visualizing annotations (...), and exporting score-aware annotations.”[5]

While modern GUI-based tools such as Sonic Visualizer are powerful, they do not always provide the flexibility required for state-of-the-art MIR research. Many research questions demand highly specialized visualisations that are not available in pre-built software, especially when multiple data types such as audio, scores, annotations, and computational outputs must be combined in a single workflow. In these cases, programming-based methods are often necessary.

Code-based workflows are also attractive because they can be versioned, shared, and included directly in publications, which improves reproducibility. This is especially important in academic research, where results must be inspectable and repeatable. In addition, MIR increasingly overlaps with machine learning and large-scale data analysis, which naturally favours scripted workflows for feature extraction, training, evaluation, and custom visualisation.

Currently, Python is the go-to language for much of MIR research. Libraries such as Librosa, Matplotlib, and Madmom, together with tools such as Jupyter Notebooks, are commonly used to implement these workflows.

Taken together, these reasons explain why code-based visualisation remains essential in MIR and why Piano Precision is a useful motivating example for this thesis: it sits precisely at the intersection of interactive GUI support, audio-to-score alignment, and reproducible computational workflows.

2.3 Audio-to-Score Alignment

Audio-to-Score Alignment, also referred to as score matching or score alignment, consists of synchronizing an audio recording of a musical performance with its corresponding symbolic score [2]. In practical terms, the goal is to establish correspondences between what is heard in the recording and what is represented in the score, enabling tasks such as score following, guided listening, interactive navigation between audio and score, and research-oriented analysis.

In the literature, it is useful to distinguish between two broad alignment strategies. The first is **time-warped alignment**, in which the score is mapped onto a continuous performance time-line. This approach preserves a direct temporal relationship between score elements and audio playback, and it is especially relevant when every relevant musical event needs to be placed along the time axis. The second is **link-based alignment**, in which the relationship between score and performance is established through discrete anchor correspondences. Rather than continuously warping the score, this approach links selected score positions to performance positions and uses those reference points to organize the alignment.

These two strategies are not separate tasks so much as different ways of representing the same underlying correspondence problem, and the distinction becomes important for the rest of this thesis. Time-warped alignment is the more continuous formulation, while link-based alignment is the more anchor-driven formulation. This terminology will be used throughout the following chapters.

Alignment systems can also be classified by when the matching is performed. **Offline alignment** processes a complete recording after the performance has finished. Because it can access the full signal, offline alignment can use future information to determine the best possible correspondence, which makes it suitable for analysis, annotation, and audio editing. **Online alignment**, also known as score following, performs the matching in real time while the audio is being received. Since it cannot rely on future information in the same way, online alignment typically has to trade off latency against precision.

Beyond this temporal distinction, alignments may be carried out at different levels of granularity. The three most common are **note-level**, **beat-level**, and **segment-level** alignment.

Note-level alignments are score matching processes that aim to synchronize at the individual note level. At its core, note-level alignments are systems based on note and onset/offset detection. In Music Information Retrieval (**MIR**) literature it can be also referred as “fine-grained alignment” due to the level of precision that these methods require. Contemporary Audio-to-Score **MIR** research focuses on this type of alignment not only because of the precision element but also because it can retrieve data that is relevant for other **MIR** fields and tasks. However, pure note-level alignments might falter if the performance deviates from the score (i.e. if the performer skips, repeats or adds notes or sections of the score). That’s why some score matching solutions try to employ, on top of fine-grained alignments, other methods.

Piano Precision is an example of an application of a link-based note-level score-alignment method. While the recording is playing Piano Precision shows a spectrogram and waveform that follow along a conventional playback cursor while, on a side window that contains the score, highlighting the corresponding elements being played in the score, note by note. This enables annotations to be mapped to both the audio and individual score elements simultaneously. The authors note the usefulness of this tool for performance researchers [5] however, we will make the case that such feature is useful for BT and MIR research more broadly.

Beat-Level alignments aim to follow along beat and/or measures dictated in the score, and then linking them to the performance. Early score aligners [3] would rely on such methods. Today,

score alignment solutions, although mostly are note-level types, still combine Beat-Level alignment layers within their algorithms. A good example of this comes from [12], in it the authors improved on other alignment models, one strategy employed was adding analysis of “alignments at both the beat and the bar level”. The beat level alignment would serve as “anchor points” in order to prevent “cascading errors, as misalignment will not propagate beyond the next anchor point”, they noted that this improved results of automatic alignments.

Segment type alignments, map broad portions of performance audio to corresponding visual blocks in the score. This type of score matching is mostly made manually, it is common to see it in follow-along videos to help viewers listen to the music while visualizing the score. This type of alignment is considered “weak but musically meaningful” [6] since segments can be deliberately chosen i.e., to better show melodic or harmonic structure.

Among the visual alignment tools relevant to this thesis, ScoreWarp is especially important. ScoreWarp presents a score that is tied to a physical time axis, where each musical element is shifted so that its visual position coincides with the time at which it sounds [4]. In other words, it implements a time-warped visualisation of the score while preserving the semantic structure of the original MEI data. This makes ScoreWarp a strong example of how alignment can be used not only as an algorithmic task, but also as a visual framework for interacting with music information. In the next chapter, we will return to this tool in more detail, since it became one of the key references for the development of this thesis.

2.4 Summary

This chapter provided a comprehensive overview of **MIR** visualization techniques and tools, establishing the foundation for understanding audio-to-score alignment.

We began by categorizing music visualizations into three main types: traditional sheet music representations, symbolic machine-readable formats, and audio-based representations. With this we established the importance of the Music Encoding Initiative (**MEI**) and Verovio as well as the usage of the Scalable Vector Graphics (**SVG**) visual format in this context.

We then examined the contemporary landscape of **MIR** visualization tools, distinguishing between **GUI**-based solutions such as **SV!** (**SV!**) and **PP!** (**PP!**), which provide user interfaces for audio analysis, and code-based workflows that afford researchers greater flexibility to develop specialized visualizations tailored to specific research questions. Python-based approaches using libraries like Librosa and Matplotlib have become the de facto standard for **MIR** research, particularly due to their integration with machine learning pipelines and reproducibility requirements.

Finally, we introduced one of the core tasks of this thesis: Audio-to-Score Alignment. This process synchronizes musical performances with their corresponding symbolic scores. ScoreWarp emerged as a particularly relevant tool, demonstrating how **MEI** and Verovio can be leveraged to create visually warped scores that maintain semantic integrity while aligning with performance time

Chapter 3

Methodology

This chapter presents the methodological choices that guided the development of this thesis. It explains the overall research approach, the iterative process used to build and refine the work, the scope boundaries that define what the project does and does not cover, and the criteria used to evaluate the end result.

3.1 Research Approach

This thesis follows a design-oriented and implementation-driven research approach, supported by iterative prototyping. Rather than treating the problem as a purely theoretical exercise, the work focuses on building and refining a practical framework for time-aligned music visualisation that responds to the needs identified in the literature review and in the reference tools analysed in the previous chapter. The research process is therefore grounded in making, testing, and adjusting concrete visual and interaction decisions as the project evolves.

The methodology combines conceptual analysis with software development. First, the problem space was studied through the comparison of existing tools, visualisation strategies, and alignment workflows. That understanding was then used to define a focused system design that could support layered, time-aligned representations of musical data. From that point onward, implementation and evaluation proceeded together: each prototype was used to verify whether the proposed interaction model, visual structure, and alignment logic were adequate for the thesis goals.

This approach is appropriate because the main contribution of the thesis is not only to describe a concept, but to demonstrate how such a concept can be materialised into a workable framework. As a result, the methodology emphasizes practical feasibility, visual clarity, and alignment consistency, while still remaining informed by the academic context established in the literature review.

3.2 Iterative Development Process

The development of LayerIt followed an iterative path made up of five prototype cycles before the final framework took shape. Rather than moving from a fixed specification directly to implementation, the project evolved through successive experiments in which each prototype was built to answer a specific question, evaluated for what it revealed, and then either discarded or used as the basis for the next step. Seen in this way, the value of the iterations is not only in the software they produced, but in the design knowledge they accumulated.

We started by creating **BeatMapJS**. Its goal was to explore a web-based beat-annotation interface and to test whether existing beat-tracking tools could be combined with a more interactive annotation workflow. The prototype was built in JavaScript and HTML using BeatThis and Wavesurfer.js, and it included an interactive spectrogram, region annotation, a tempo-annotation table, and import/export support for SV-compatible text files. The main lesson was that, although the system was functional, it did not contribute a meaningful new annotation capability beyond the integration of BeatThis. More importantly, it did not produce a reusable Python library, which, from the start, was an important requirement. The decision carried forward was that a reusable Python core would be mandatory.

The second cycle was **MEI-Basic-Edit**. Its goal was to test whether manual score alignment could support audio-to-score annotation by allowing score elements to be adjusted against an audio layer. The prototype was built in JavaScript and HTML with Verovio, MEI, and SVG, and it rendered a score below a static spectrogram while allowing users to drag score elements horizontally to align them with the audio representation. It also included an embedded XML editor for direct manipulation of the underlying score code. The key lesson was that interactive score editing was expensive both conceptually and technically: it increased UI complexity and required a level of implementation effort that was not justified by the results. At the same time, this cycle provided valuable familiarity with MEI, Verovio, and SVG structure. The decision carried forward was that SVG would remain a viable composition substrate, but interactive editing would not be the main direction.

The third cycle was **BeatPrecision**, a fork of Piano Precision. Its goal was to extend an existing GUI environment with beat-annotation features while preserving the score-alignment functionality already present in the software. The prototype was built in Qt/C++, integrating BeatThis and implementing horizontal score-alignment alongside the existing spectrogram view. While functional, the project revealed significant challenges. Piano Precision's (and by extension Sonic Visualizer's) codebase proved complex and difficult to manage. Long compile times and the sheer size of the codebase made each developed modification increasingly challenging to implement, manage, and debug. More importantly, this iteration taught us a critical lesson: the need to distinguish between the interaction and visualization problems in BA. We realized we could not tackle both simultaneously and would need to choose a direction. Recognizing a clear gap in the field, we opted to focus on the visualization aspect, as we saw greater potential for the thesis to benefit the community in this area.

The fourth cycle was **Score-Alignment.py**. Its goal was to explore score-aligned spectrograms in a Python environment and to assess the Modusa framework as a possible basis for the project. The prototype was built in Python with Modusa and Matplotlib and produced an interactive spectrogram. The lesson was that a truly fluid interactive visualization, with zooming, panning, a timeline, and playback control, was much harder to implement than expected. Even though the prototype worked, the result was not satisfactory enough to justify continuing in that direction, and the return on effort was not viable. The decision carried forward was to abandon real-time interactivity and instead prioritize static, high-quality, reproducible output.

The fifth cycle was **Score-Alignment.js**. Its goal was to revisit the aligned-spectrogram idea using the lessons learned from the earlier MEI and beat-annotation prototypes. The prototype was built in JavaScript and HTML with Verovio, MEI, and Wavesurfer.js, and it displayed a MEI score alongside a spectrogram while storing beat timestamps in the MEI data and highlighting notes on interaction. The lesson from this final exploratory cycle was that the idea was promising, but it also confirmed the need for a tool that was strictly focused on visualization, written in Python, and organized in an object-oriented way so that it could be reused and extended more easily. The decision carried forward was that this combination of requirements defined what would then be LayerIt!.

Table 3.1: Summary of development iterations

Iteration	Goal	Tools Used	Lesson	Conclusion
BeatMapJS	Explore web-based beat annotation	JS/HTML, BeatThis, Wavesurfer.js, interactive spectrogram, region annotation, tempo table, SV-compatible import/export	No new annotation feature beyond BeatThis; no reusable Python library	Reusable Python core is mandatory
MEI-Basic-Edit	Test manual score alignment against audio	JS/HTML, Verovio, MEI, SVG, drag-to-align score, XML editor	Interactive score editing is costly; gained MEI/Verovio/SVG familiarity	SVG as composition substrate; no interactive editing focus
BeatPrecision	Extend GUI beat annotation in an existing score-aligned tool	Qt/C++, BeatThis, horizontal score follow-along	Codebase was hard to extend; interaction and visualization must be separated	Focus on visualization, not the interaction problem
Score-Alignment.py	Explore score-aligned spectrograms in Python	Python, Modusa, matplotlib, interactive spectrogram	Fluid interactivity proved impractical	Prefer static, high-quality, reproducible output
Score-Alignment.js	Revisit aligned spectrograms with richer score interaction	JS/HTML, Verovio, MEI, Wavesurfer.js, beat timestamps in MEI, note highlighting	Promising, but confirmed need for a Python OO visualization tool	Define the LayerIt! conceptual baseline

Taken together, these five cycles show a deliberate narrowing of scope. The project moved from an interactive beat-annotation GUI toward a reusable, object-oriented visualisation frame-

work. Each prototype clarified one part of the problem and ruled out another, so the final system could be built on a more stable understanding of what the thesis needed to solve.

3.3 Scope Decision

The development process described above led to an important scope decision. At the outset, the project was framed broadly as a possible BA/BT graphical user interface, with both annotation and visualisation treated as potential contributions. However, the iterative prototypes made it clear that these were, in practice, two different problems, each with its own technical demands, design goals, and implementation costs. Attempting to solve both within a single master's thesis would have diluted the contribution and made it harder to reach a result that was both robust and meaningful.

For that reason, the project deliberately narrowed its focus from an interactive BA/BT GUI to an OO visualisation framework. The interaction problem proved too broad to address properly alongside the visualisation problem, while the visualisation side revealed a clearer and more relevant gap: the lack of a reusable, flexible, and well-structured Python tool for producing layered time-aligned musical data visualisations. By focusing on this gap, the thesis could concentrate on a contribution that was both feasible and valuable.

As a result, this thesis does not attempt to provide a complete BA/BT annotation environment, nor does it seek to solve the broader interaction problem in full. Instead, it focuses on the visualisation dimension, with particular emphasis on clarity, modularity, and alignment consistency. This narrower scope makes it possible to develop a more coherent contribution and to evaluate it against well-defined criteria.

3.4 Evaluation Criteria

The evaluation of this thesis is guided by a small set of criteria chosen to reflect the actual purpose of the framework. Since the goal is not to produce a general-purpose MIR platform, but rather a reusable visualisation tool, the evaluation focuses on whether the final system succeeds in covering the visualisation cases that motivated its development, while keeping the implementation manageable and the output reliable. In this sense, the framework is judged less as a standalone demo and more as a deliberate response to a specific research and design problem.

Firstly, the framework should be able to represent the kinds of score-aligned visualisations required by the thesis, including the layered combination of symbolic score data with audio-derived and annotation-derived information. Secondly, the reduction of manual effort. One of the motivations for the work was to avoid *ad hoc* image editing and repetitive figure construction, so the framework should make the creation of complex visualisations faster, clearer, and less dependent on manual intervention. Thirdly, concerns reproducibility. Because the thesis is situated in a research context, the visual outputs should be generated through a process that can be repeated, inspected, and shared without ambiguity.

Two further criteria are extensibility and fidelity. Extensibility refers to whether the framework can be adapted to new visual layers or new combinations of existing ones without requiring major restructuring. Fidelity refers to whether the resulting figures preserve the musical and temporal relationships they are intended to represent, while still making clear the limits of the visual abstraction. In other words, the evaluation must consider not only what the framework can display, but also where its representation becomes less precise, less general, or less appropriate. Defining these criteria here makes the evaluation of the thesis read as a deliberate design assessment rather than as a mere presentation of the implemented tool.

Chapter 4

LayerIt!

Design and Implementation

This chapter presents LayerIt as the concrete outcome of the research process described in the previous chapter. Its role is to show how the thesis moves from a problem statement and a set of methodological decisions to a working framework for composing score-aligned MIR visualizations. The chapter is organized from the problem definition to the system overview, then to the design choices that shaped the implementation, and finally to the mechanism used to combine the different visual layers into a single result.

4.1 The Problem and our Solution

The central problem addressed by LayerIt is that score-aligned MIR figures are often created through a mixture of manual editing, *ad hoc* scripting, and tool-specific workarounds. In practice, this makes it difficult to produce figures that are reusable, reproducible, and easy to adapt when the underlying data changes. Existing GUI tools are useful for inspection and annotation, but they are not designed to support the kind of compositional, publication-oriented visual output required by this thesis.

LayerIt responds to this problem by treating visual elements as composable layers rather than as a single fixed image. The framework provides a structured way to generate, combine, and align music-related visual materials in SVG-based output. The alignment backbone is provided by ScoreWarp. The contribution of LayerIt is the object-oriented layer-composition model that makes the visual construction process more flexible and more reusable.

4.2 LayerIt Overview

LayerIt is a Python framework designed to compose score-aligned MIR visualizations from independent visual components. At a high level, it takes aligned musical data, generates the corresponding visual representations, and arranges them into a layered figure that preserves the tem-

poral relationship between the score and the accompanying audio-derived or annotation-derived information.

The framework is modular, this means that the same base structure can support different combinations of score material, spectrograms, annotations, beat information, or other derived data without requiring the entire figure to be rebuilt manually. The result is a visualization workflow that is easier to extend than a static editing process and more reproducible than a one-off graphical composition.

The project bases itself in librosa [8], matplotlib [14] and modusa [9] to generate audio and data visualizations. The score layer is rendered through ScoreWarp. The end result is an SVG image file that overlays the audio and data layers, with the score. The codebase also integrates BeatThis the state-of-the-art BT algorithm. Through its implementation we demonstrate how a given BT model can be integrated to our system in order to produce beat data visualizations.

4.3 Design Choices and Rationale

LayerIt! relies on matplotlib for audio and data representations where, the SVG format is the end visual format. The reason, stems from the fact that Verovio (and subsequently ScoreWarp) render the score in SVG, and because matplotlib does not provide seamless integration of SVG charts, it is more reliable to export matplotlib visuals into an SVG framework, then the other way around.

The SVG format, is a vector-based one, therefore, data that can be mathematically represented in cartesian plots (i.e., X/Y graphs) can be exported without losing resolution. This is the case for most of MIR algorithmic data i.e., continuous curves and time based annotations. However, some data types, cant be presented in this way noticeably, heatmap-based plots like spectrograms. For interaction proposes, these usually are rendered in real-time, allowing for seamless navigation (i.e., zooming and panning). This is not possible since SVG is an inherently static format, therefore plots like these need to be rasterized before being integrated into a SVG visualization. To circumvent this, we employed a way to render plots as PNG's as well. This trade-off means that cartesian plots and the score can be zoomed-in infinitely while always preserving the highest level of fidelity and accuracy. On the other hand, rasterized layers are bound to lose quality with zooming while, at the same time, also requiring heavier processing and memory usage.

That being said, because SVG is a text markup format, it can be easily accessed and modified, allowing it to be an intermediate format that can bridge different codebases independently of their base language (i.e., Python, JS, C++ etc...). This also serves as a crucial value argument for this choice, it being: the development and integration of LayerIt into future MIR applications.

Our main goal is that, this codebase may be easily integrated into other relevant projects to generate visualizations. Hence why we developed it as a Python Based tool that is based on commonly used MIR libraries such as librosa and matplotlib. This way, community integration and development is eased.

4.4 Generating Visuals

LayerIt was developed with the goal of generating visualizations for BA and BT tasks, therefore, the current version focuses on tools for these purposes. Hence the reason we focus so much in providing tools for audio-to-score and data alignment tools, since we believe that this is a core gap for annotation and evaluation tasks for BT and BA tasks and subsequently for MIR as a whole. In its current state the codebase relies on the following files to generate complete visuals:

File	Description
.wav	File with the audio recording of the performance
.mei	XML Score of the musical piece
.maps.json	Contains data that links onsets with score elements of the .mei
.svg	The score in visual format generated by ScoreWarp (requires .mei and .maps <i>a priori</i>)
.npz	File with algorithmic BT data

Table 4.1: Files required by LayerIt to generate complete visuals.

4.4.1 Getting the Warped Score

The warped score can be generated using the [ScoreWarp online demo](#). In it, one needs to upload the score in .mei format and the MAPS file in order to download the score in svg format.

The .mei file that contains the score in xml format. The easiest way to get any .mei score is to get the score in .musicxml format and then convert it to .mei, this can be done in multiple ways we will describe two: The first being through the [verovio online editor](#). The second, by using musescore’s software that allows to then convert to .mei. In either way, it is important to get the original score in .musicxml format to be able to then convert it to .mei using the tools we described.

4.4.2 Getting the MAPS file

The MAPS file, describes the onsets of the performance and links them to each corresponding score elements of the .mei score through their @xml.id . Getting the maps file might be a bit more cumbersome, but we can describe two different ways of obtaining it:

The first is the one described in [4], using the MIDI-to-MIDI matching algorithm by [11]. However, this method requires , along side with the .mei score, the performance in midi format. Using examples from the asap dataset [12] will facilitate this process, but if a midi performance is not possible to obtain then the maps file needs to be generated independently.

The MAPS file is at its core a .json file, and for the purposes of the current LayerIt version, it needs to provide the score’s and performance onset data in the following manner:

Listing 4.1: Minimal structure of the MAPS.json file to work with LayerIt and ScoreWarp

```
1 {  
2   "obs_mean_onset": 0.853333333,  
3   "xml_id": ["x1iw1eq1"],  
4   "obs_num": 1  
5 }
```

Where:

- “*obs_mean_onset*” is the time in seconds of a given onset.
- “*xml_id*” is the @xml.id of the corresponding onset note in the .mei score. In the case that the onset has more than one note, like a chord hit, this attribute can have more than one @xml.id.
- “*obs_num*” acts as a simple counter for each provided onset.

LayerIt relies on MAPS in two complementary ways: directly for onset visualization, and indirectly through ScoreWarp, which requires MAPS to generate the warped score aligned with the timeline SVG element which constitutes the alignment reference for all overlaid data.

ScoreWarp performs best when MAPS contains dense onset annotations across the full performance. However, it can still align scores when annotations are sparse via interpolation between available onsets and their @xml.id anchor points. In practice, this allows different precision levels (i.e., downbeats, phrase boundaries, or measure-level anchors), trading annotation effort for alignment precision. Even minimal anchors (first and last onset) may be usable for short performances or segments, but intermediate alignment becomes less reliable due to interpolation error.

For the examples that we provide in the making of our codebase we modified PP to obtain this data. PP’s “Piano Aligner” plugin [5] matches performance onsets to score elements defined by the .mei file. In PP these are displayed in a regular SV time instance layer. We made a simple modification where, this layer, instead of displaying each onset with it’s corresponding score position, “*measure + position_in_measure*”, it would display the @xml.id of each onset. With this we could then export the time instances as .txt files with “*obs_mean_onset+xml_id*”, then convert it into the MAPS format using “*TXT_to_Maps()*” function in @src/functions/warp_score.

4.4.3 Providing BT data

Since contemporary BT algorithms are ML based, they primarily function by generating a beat activation function, which is a continuous curve representing the probability of a beat occurring at any given frame. Visualizing this underlying probability curve, rather than just the discrete beat assessments, allows us to see the algorithm’s internal confidence level, giving crucial insights on the inner workings of the model.

In @src/functions/Beat_Layers, the function “*Run_BeatThis()*” generates an .npz file that contains the required data for this to be later visualized. This particular function creates the .npz file generated through BeatThis. However, the .npz file format can serve as an intermediate for BT

visual layers, as long as it is structured like it is described in “*Run_BeatThis()*”. The .npz should contain five key components from the BT model:

Field	Purpose
beat_times	Time coordinates (seconds) aligned with probability frames
beat_probs	Continuous beat activation function values
downbeat_probs	Continuous downbeat activation function values
detected_beats	Discrete beat positions derived from peak detection
detected_downbeats	Discrete downbeat positions from peak detection

The probabilistic fields capture the algorithm’s internal confidence across time, while the detected positions represent the final beat assessments. This separation enables visualization of both the continuous confidence landscape and discrete beat estimates.

4.5 Layer Alignment Method

As of now, LayerIt provides tools for generating visualizations with BT and BA tasks in mind, however, this is not an exclusive application of the codebase. Layers (be it data or audio representations) are simply either SVG or PNG elements that span the same or a proportional width to a given timeline, therefore, users can upload any SVG or PNG plots, given that they follow the same timeline length.

When the warped score is generated, ScoreWarp provides a functionality to add a visual timeline of the performance. This timeline will most likely span a wider length than the actual recorded performance length, possibly even getting out of bounds to the visual SVG canvas. Yet, since this timeline provides the time reference for positioning the score elements correctly, it is also used in LayerIt to align different visual layers to that same recording.

The way LayerIt aligns the layers is by:

1. Looking at the SVG file containing the Warped Score (rendered through Score Warp) and retrieving the full length of the generated timeline in pixels.
2. Getting the actual audio recording duration (in time) within that same timeline.
3. Using a simple rule of three, it calculates the recording length (in pixels) by comparing the recording’s duration in time, to the full timeline length in pixels.

In the codebase this is done through a single method:

```
• Visualizer().get_Layers_WidthHeight()
```

Although LayerIt does not yet integrate methods to convert visualizations outside of what it already provides, the basic methodology within the layer creation is this. Image files only need to be placed in the same x position of the start of the related timeline and the method to get the length is already in the codebase. Any given image format, should just respect that same recording length and then scale it to the appropriate timeline length of the score generated through ScoreWarp.

4.6 Layers on Layers, within Layers

At its core, LayerIt provides a way to combine aligned visualizations of algorithmic data, audio and symbolic music representations (i.e., scores) that share the same timeline. The true value of our approach comes from the Python OO layer-based methodology that we developed in our approach. This allows not only the visualizations to be deeply customizable, but also, for implementing with other tools and workflows.

The visualizations generated by LayerIt follow a “layers within layers” philosophy: visual elements are self-contained components that can be composed into upper-level elements, which in turn can be composed into yet higher levels. [Figure 4.1](#) demonstrates this methodology.

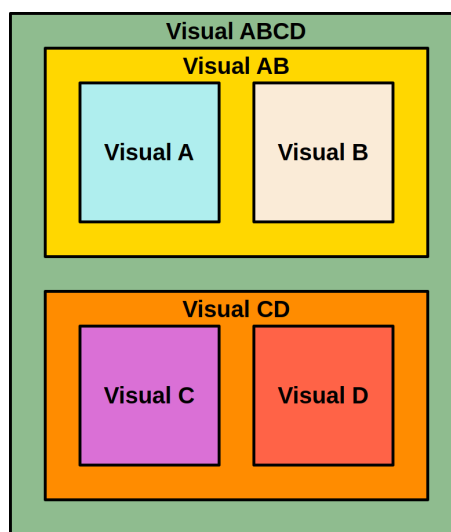


Figure 4.1: Composition of LayerIt visualization system. Each colored box represents a distinct visual element that compose higher-level visualizations.

This “layers within layers” design philosophy is applied at three different levels within the project’s codebase: the **visual level**, where visualizations are composed of aligned data, audio and musical elements; the **SVG level**, in which LayerIt organizes the XML structure of the SVG; to the **code level**, with it’s OO architecture of the codebase. The strength of this layer-based approach lies in the flexible foundation it provides, leaving the codebase open to future development and integration with other tools, workflows, and projects.

4.6.1 Visual Level

The visual level of this “layers within layers” philosophy refers to the actual visual output of LayerIt. Where layers are visual elements that are aligned within themselves and between themselves.

4.6.2 SVG Level

This refers to the way LayerIt organizes the XML structure of the SVG. Where each layer (combined or individual layers) follow a structured and organized approach. The following listing shows a simplified representation from the XML from the same SVG shown in [Figure 5.1](#):

Listing 4.2: Example structure of an SVG file

```

1 <svg>
2 |   <Top Visualization>
3 |   |   <Layers>
4 |   |   |   <image spectrogram.png>
5 |   |   |   <BeatThis Output>
6 |   |   |   <Onsets>
7 |   |   <Score>
8 |   |   <timeAxis>
9 |   <Bottom Visualization>
10 |   |   <Layers>
11 |   |   |   <image spectrogram.png>
12 |   |   |   <Beat Probability>
13 |   |   |   <Downbeat Probability>
14 |   |   <Score>
15 |   |   <timeAxis>
16 </svg>

```

This kind of XML organization makes it easier to manipulate the SVG directly through it's embedded code. This allows for another level of visualization customization (through directly changing the XML text) and also should make it more simple to be accessed via code-based solutions, easing the integration with other projects while also maintaining the SVG readability. Yet once again, we can see our layer based organization method in action, where elements are put inside each other at the XML level as well.

4.6.3 Code Level

At this level we refer to the actual OO architecture of the codebase. LayerIt's OO architecture is built on a principle of modular composability: each visualization component is a self-contained, reusable module that adheres to a consistent interface. This enables the same "layers within layers" philosophy that characterizes the visual and SVG levels to emerge naturally at the code level. The following snippets illustrate the code-level contract (base abstraction), the Visualizer composition API, and a short end-to-end workflow.

Listing 4.3 shows the 'Layer' base-class interface (illustrative). For clarity the required imports are included.

Listing 4.3: Layer interface (concise)

```
1 class Layer(ABC):
2     def __init__(self, name: str = "Layer"):
3         self._data = None
4         self.name = name
5
6     @abstractmethod
7     def load_data(self, **kwargs) -> bool:
8         ...
9
10    @abstractmethod
11    def draw(self, ax, shared_data) -> tuple:
12        ...
```

In *LayerIt!*, a visualizer instance composes multiple lower-level layers, which may represent data plots or audio representations, and provides them with a shared data dictionary via `.load_all_layers()`. The next listing (4.5) shows how a visualizer instance should be instantiated.

Listing 4.4: Visualizer Instance Composition

```
1 # Creating a visualizer and composing layers
2 Layer1 = Visualizer()
3
4 # Add multiple layers - they're composable
5 Layer1.add_layer(BeatProbabilityLayer(color=(1, 0, 0)))
6 Layer1.add_layer(DownbeatProbabilityLayer(color=(0, 0, 1)))
7 # ...
8
9 # Load all data and render the combined layers
10 Layer1.load_all_layers(audio_path="performance.wav",
11                       beat_file="beats.npz")
12 fig, ax = Layer1.draw()
```

After this, each `Visualizer()` instance can be rendered to an SVG or PNG. At this stage, `Visualizer()` outputs are still Matplotlib figures and can be displayed directly with `.show()`. However, if they are to be combined with other `Visualizer()` instances and a score, they need to be exported to SVG. This intermediate SVG layer allows the aligned score to be added on top of each visualization, producing a higher-level composite that can then be stacked with other rendered SVGs to create the final aligned view. The following listing shows exactly that.

Listing 4.5: Simplified workings for SVG layers.

```
1 # Turn the Visualizer into SVG and aligned it to the warped score
2 SVG_Layer1 = Layer1.turn_to_SVG("Output/Path/And/Filename", svg_warped_score)
```

```

3 SVG_Layer2 = Layer2.turn_to_SVG("Output/Path/And/Filename", svg_warped_score)
4
5 # Add Score to a given SVG Layer
6 SVG_Layer1_withScore = Layer1.combine_layers_with_score(filename, svg_warped_score)
7
8 # Create the final SVG with each SVG_Layer Stacked and place their relative vertical
  positions.
9 EndVisualization = Visualizer()
10 svg_layers = [(SVG_Layer1_withScore, 50), (SVG_Layer1, 200), (SVG_Layer2, 350) ...]
11 EndVisualization.create_final_SVG(width, height, (...), svg_layers)

```

This shows that higher-level SVG outputs can be positioned or stacked vertically, while lower-level layers (like Matplotlib layers) can be arranged in the same flexible way. That compositional behavior reinforces the “layers within layers” philosophy that underpins LayerIt!.

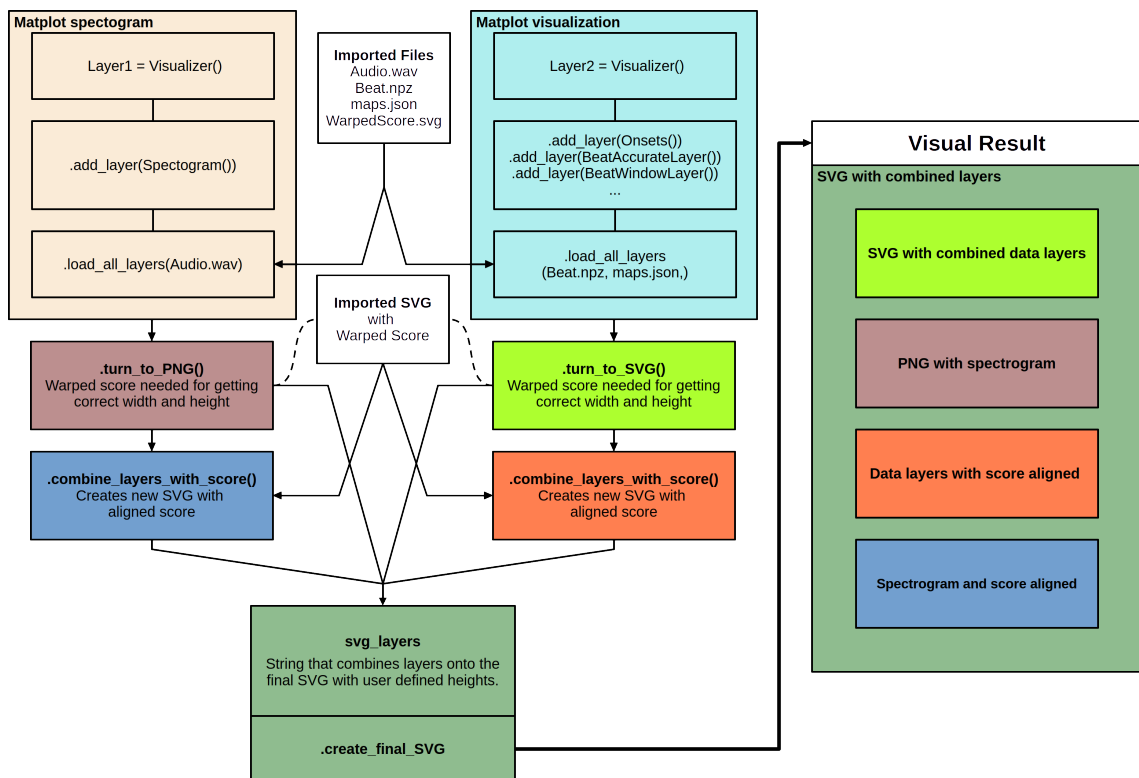


Figure 4.2: Complete LayerIt! workflow: from Visualizer() composition to final stacked SVG visualization. The diagram shows how multiple visualization layers are created independently, exported to SVG format, combined with the warped score, and finally stacked vertically with defined y-offsets to produce the final custom composite visualization.

The architecture of LayerIt! defines different levels of interaction, from matplotlib-based generation of visualizations to this SVG/PNG methodology, which adds another layer of flexibility and integration. This final layer allows users to incorporate externally produced graphics, enabling the system to combine code-generated visualizations with images created outside the codebase.

This hybrid design is intentional: it makes LayerIt useful for multiple workflows, from Python-based prototyping to GUI-centric and third-party visualization pipelines. By supporting both custom layer implementations and external image inputs, LayerIt provides a practical bridge between research code and real-world visualization needs.

These code-level choices make LayerIt more than a set of rendering tools: they make it a platform for experimentation and integration. That platform-level flexibility is what allows new visualization types to be added without changes to the core framework, supporting the extensibility requirements outlined in ?? and positioning LayerIt as a foundation for future MIR visualization work.

4.7 Summary

This chapter closes the methodological and implementation stages of the thesis by connecting the development process described in Chapter 3 with the framework described in this chapter. The iterative path taken through the five prototypes clarified the scope of the work, showing that the most meaningful contribution was not a new annotation system, but a reusable visualization framework grounded in time-aligned music data.

LayerIt was then shaped to answer that need: a Python, object-oriented framework for composing score-aligned MIR visualizations through layered SVG-based output. Its design emphasizes reuse, composability, and visual consistency, while relying on ScoreWarp for the alignment backbone and on established MIR libraries for the generation of the underlying audio and data layers. Taken together, the two chapters show how the project moved from exploratory prototyping to a focused implementation that can now be assessed against the criteria established in the methodology chapter.

Chapter 5

Validation

5.1 Validation Approach

5.2 Reproducing Composite Figure Types (RQ1)

5.3 Workflow Comparison: Annotation and Evaluation (RQ2, RQ3)

===== Section INCOMPLETE/UNREVISED =====

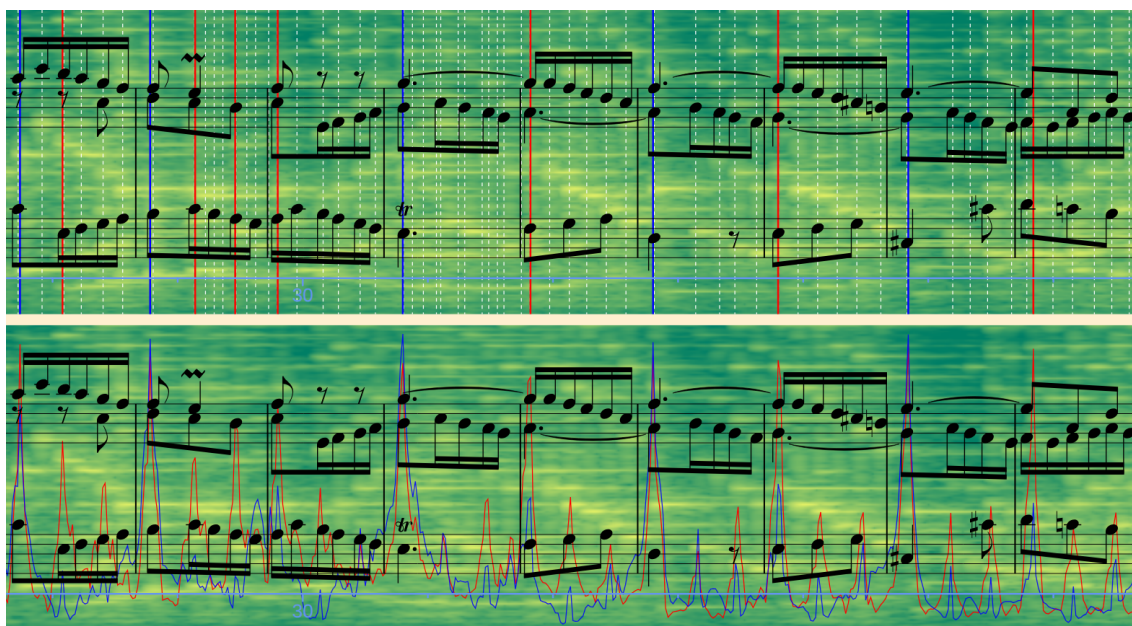


Figure 5.1: LayerIt generated visualization example for comparing the end output of BeatThis (on top) and the probability layer (at the bottom). In blue downbeats, in red beats and the dotted white lines show onsets.

In [Figure 5.1](#) we can see two distinct visualizations of the same recording. Both have distinct data layers that compose the top and bottom visualizations of the same performance, the top one provides onset and BeatThis unaltered output. The bottom one, shows BeatThis assessment of a

frame-by-frame curve with the beat positions probability. Visually we can identify multiple layers that compose the individual visualizations: the spectrogram layers, score layers and respective data layers, we can also identify the two upper-level aligned layers that compose the whole example image.

Figure 5.1 is a particularly good example that demonstrates the capability of this kind of visualization for annotation and evaluation for BT purposes. On the top example we can see that BeatThis misses some beats and downbeats. However, on the bottom one, we can see that the model actually shows probability peaks that widely correspond to the actual beat positions. This type of visualization shows a concrete example for evaluating (in this case) the BeatThis model that SV simply cant provide in this way. For annotation purposes we could use this kind of visualization to effectively see missing or wrongly annotated beats/downbeats.

5.4 Extensibility Probe (RQ2)

5.5 Limitations and Difficult Cases (RQ2, RQ3)

5.6 Summary

Chapter 6

Conclusions and Future Work

6.1 Achievements

6.2 Future Work

6.3 Final Remarks

This thesis began with the observation of a critical gap in the MIR research community: the absence of a unified framework for generating time-aligned, multi-modal visualizations suited to contemporary research workflows. Through an iterative development process and careful architectural design, we have delivered LayerIt, a Python-based, object-oriented tool that addresses this gap while explicitly supporting future extension and integration.

LayerIt’s value lies not merely in its immediate applicability to beat tracking and annotation tasks, but in its demonstration that visualization frameworks can be architected for extensibility, composability, and community adoption. By grounding the system in established conventions (OO design patterns, common MIR libraries, standardized formats), we have created a foundation upon which the community can build up on.

As MIR research continues to evolve, driven by advances in machine learning, increasing dataset complexity, and growing interdisciplinary collaboration. The need for sophisticated, flexible visualization tools will only intensify. LayerIt is positioned to serve as a foundation for this future work, enabling researchers to focus on their core innovations rather than reimplementing visualization infrastructure from scratch.

References

- [1] *An Introduction to MEI*. URL: <https://music-encoding.org/about/>.
- [2] J J Carabias-Orti et al. “AN AUDIO TO SCORE ALIGNMENT FRAMEWORK USING SPECTRAL FACTORIZATION AND DYNAMIC TIME WARPING”. In: ().
- [3] Zhiyao Duan and Bryan Pardo. “Soundprism: An Online System for Score-Informed Source Separation of Music Audio”. In: *IEEE Journal of Selected Topics in Signal Processing* 5.6 (Oct. 2011), pp. 1205–1215. ISSN: 1932-4553, 1941-0484. DOI: [10.1109/JSTSP.2011.2159701](https://doi.org/10.1109/JSTSP.2011.2159701). URL: <http://ieeexplore.ieee.org/document/5887382/> (visited on 06/07/2026).
- [4] Werner Goebel. “Let’s Do the ScoreWarp Again! Shifting Notes to Performance Timelines”. In: (2025).
- [5] Yucong Jiang and Werner Goebel. “PIANO PRECISION: ADVANCING MUSIC PERFORMANCE ANALYSIS BY INTEGRATING USER-CORRECTABLE AUDIO-TO-SCORE ALIGNMENT”. In: (2025).
- [6] Jongmin Jung et al. “Unified Cross-modal Translation of Score Images, Symbolic Music, and Performance Audio”. In: *IEEE Transactions on Audio, Speech and Language Processing* 34 (2026), pp. 1876–1891. ISSN: 2998-4173. DOI: [10.1109/TASLPRO.2025.3648794](https://doi.org/10.1109/TASLPRO.2025.3648794). arXiv: [2505.12863](https://arxiv.org/abs/2505.12863) [cs.SD]. URL: <http://arxiv.org/abs/2505.12863> (visited on 06/02/2026).
- [7] Zulfidin Khodzhaev. “A Practical Guide to Spectrogram Analysis for Audio Signal Processing”. In: ().
- [8] Brian McFee et al. *Librosa/Librosa: 0.11.0*. Version 0.11.0. Zenodo, Mar. 11, 2025. DOI: [10.5281/ZENODO.15006942](https://doi.org/10.5281/ZENODO.15006942). URL: <https://zenodo.org/doi/10.5281/zenodo.15006942> (visited on 06/22/2026).
- [9] meluron. *Modusa*. Version 6.3.7. Feb. 2026. URL: <https://github.com/meluron/modusa>.
- [10] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Cham: Springer International Publishing, 2015. ISBN: 978-3-319-21944-8 978-3-319-21945-5. DOI: [10.1007/978-3-319-21945-5](https://doi.org/10.1007/978-3-319-21945-5). URL: <https://link.springer.com/10.1007/978-3-319-21945-5> (visited on 06/02/2026).
- [11] Eita Nakamura, Kazuyoshi Yoshii, and Haruhiro Katayose. “PERFORMANCE ERROR DETECTION AND POST-PROCESSING FOR FAST AND ACCURATE SYMBOLIC MUSIC ALIGNMENT”. In: ().

- [12] Silvan David Peter et al. “Automatic Note-Level Score-to-Performance Alignments in the ASAP Dataset”. In: *Transactions of the International Society for Music Information Retrieval* 6.1 (June 26, 2023), pp. 27–42. ISSN: 2514-3298. DOI: [10.5334/tismir.149](https://doi.org/10.5334/tismir.149). URL: <http://transactions.ismir.net/articles/10.5334/tismir.149/> (visited on 06/07/2026).
- [13] Répertoire International des Sources Musicales. *Verovio Reference Book*. 2021. DOI: [10.5448/7EM6-MY23](https://doi.org/10.5448/7EM6-MY23). URL: <https://book.verovio.org/> (visited on 06/06/2026). Pre-published.
- [14] The Matplotlib Development Team. *Matplotlib: Visualization with Python*. Version v3.10.9. Zenodo, Apr. 24, 2026. DOI: [10.5281/ZENODO.19716234](https://doi.org/10.5281/ZENODO.19716234). URL: <https://zenodo.org/doi/10.5281/zenodo.19716234> (visited on 06/22/2026).
- [15] Rafael Valle. “Visual Display and Retrieval of Music Information”. In: ().